Таксопарк

Над проектом работали

- Денис Колодочка
- Антон Круглышев
- Владислав Сафонов

О проекте

- Создание и обработка заказов
- Интеллектуальное назначение водителей
- Управление доступностью водителей
- Финансовый учет и рейтинговая система
- Интерфейсы для клиентов, водителей и владельца

Жизненный цикл

Создание

Назначение водителя

Выполнение

Завершение

Стратегия распределения нагрузки в час пик

Архитектура системы

Класс	Ответственность
TaxiPark	Ядро системы, управляет всеми процессами
Driver	Хранит данные водителя и его статус
Order	Инкапсулирует данные заказа
MapPoint	Географические точки (адреса)
AssignmentStrategy	Алгоритм выбора водителя для заказа
OrderSubject	Реализация паттерна Наблюдатель
ClientInterface	Интерфейс для клиентов
DriverInterface	Интерфейс для водителей
OwnerInterface	Интерфейс для владельца

Ключевые взаимодействия

Клиент создает заказ через **ClientInterface**

Таксопарк использует **стратегию** для выбора водителя

Уведомление водителя через паттерн **Наблюдатель**

Водитель завершает заказ через **DriverInterface**

Таксопарк обновляет статус и проверяет **очередь** заказов

Владелец получает отчет через **OwnerInterface**

Паттерны

- **Стратегия** - DriverAssignmentStrategy
- **Наблюдатель** - OrderSubject
- **Одиночка** - DriverAssignmentStrategy
- **Интерфейс** - ClientInterface, DriverInterface, OwnerInterface

Возможные улучшения:

Интеграция с картографическими API:

- Реальное расстояние вместо линейной аппроксимации
- Учет дорожной ситуации

Система оплаты:

- Интеграция с платежными системами
- История транзакций

Расширение функционала:

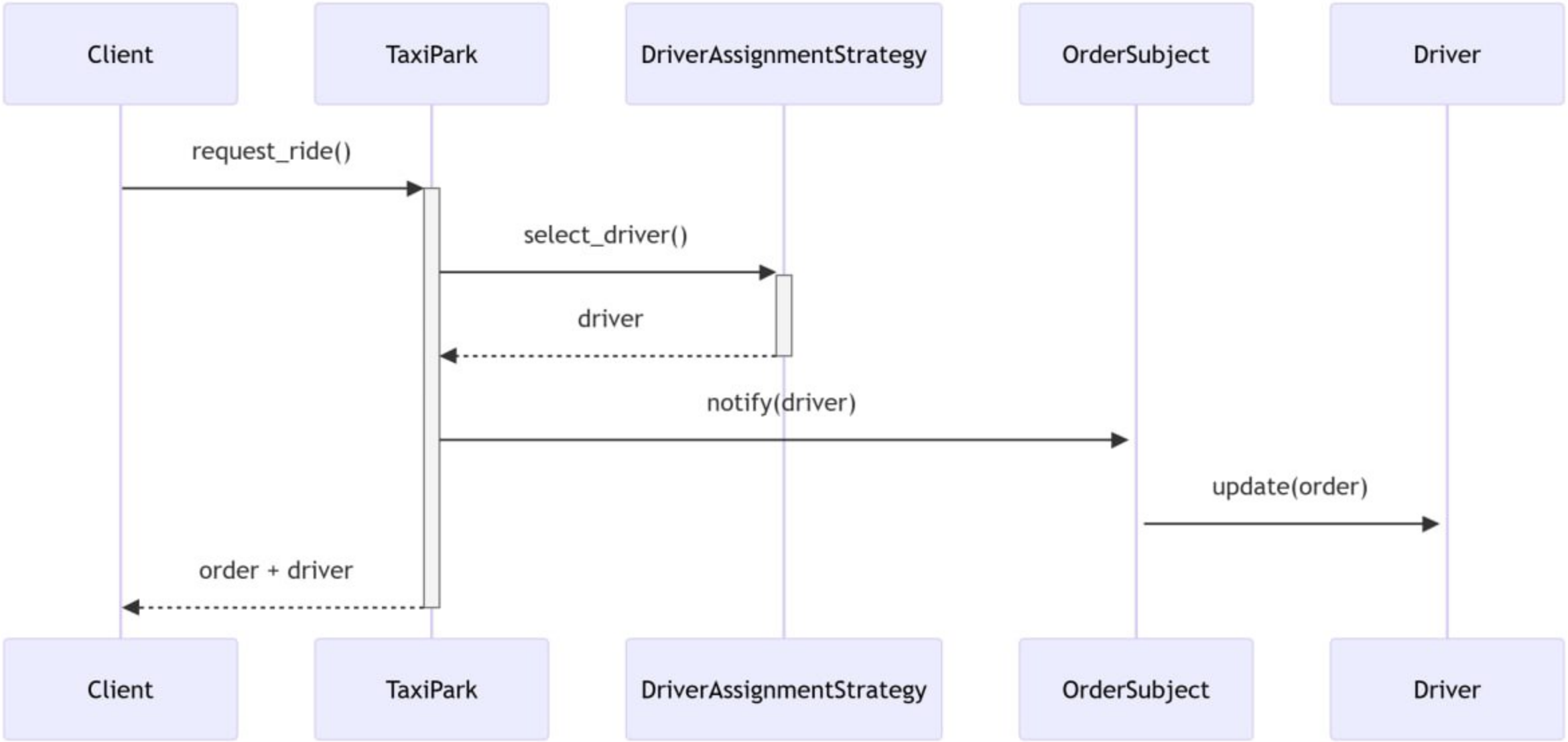
- Отмена заказов
- Оценка поездки клиентом

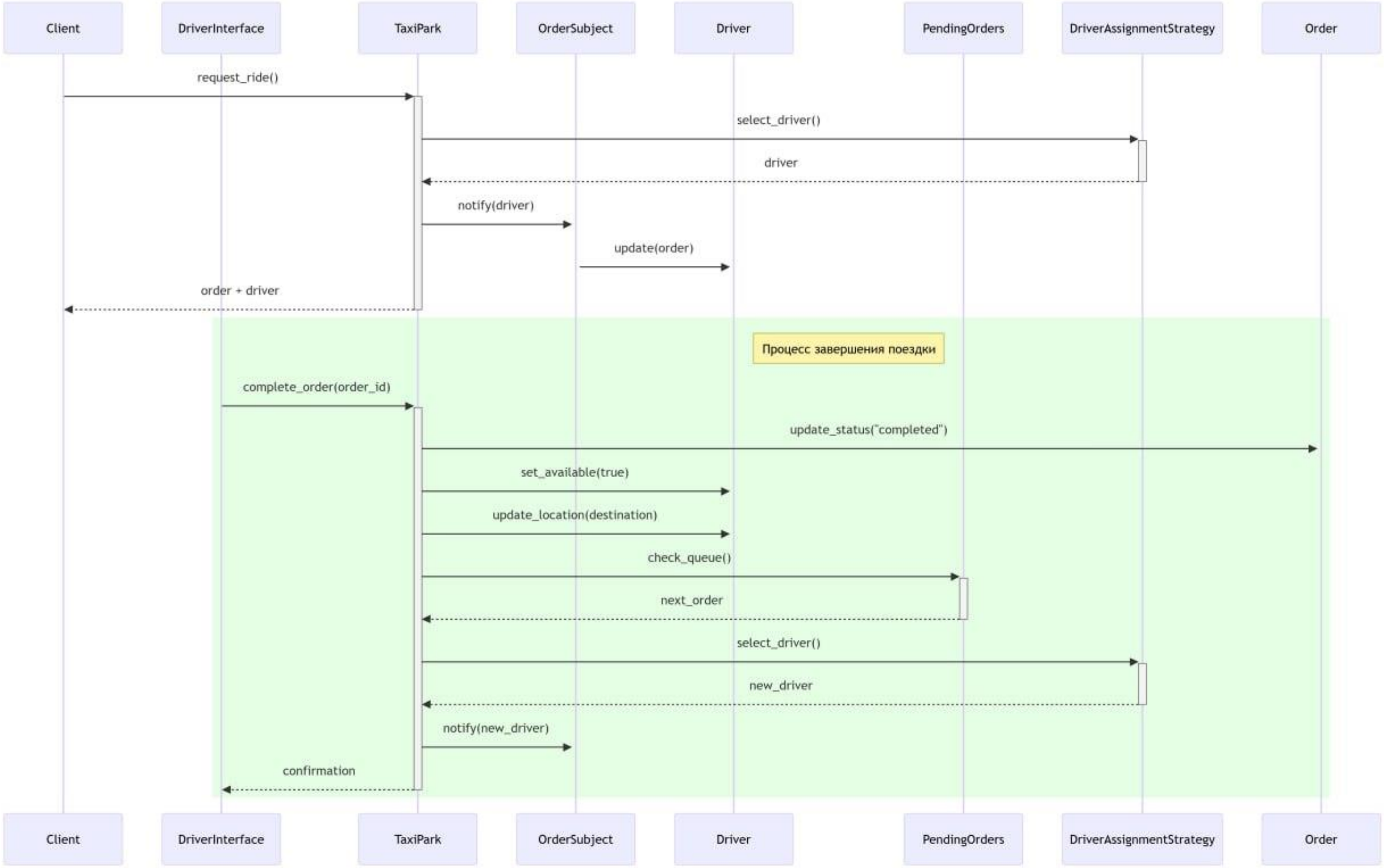
Масштабируемость:

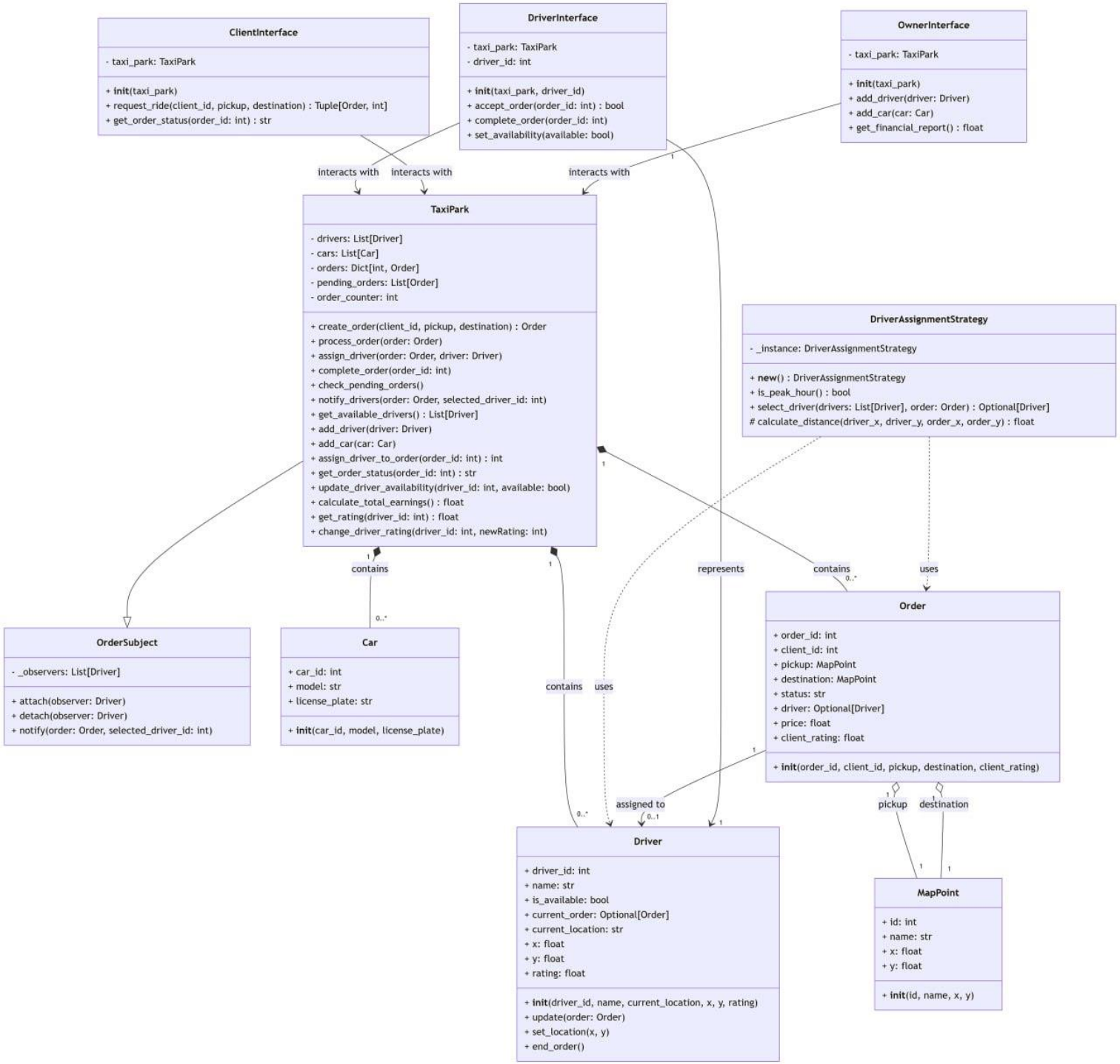
- Микросервисная архитектура
- Кеширование данных

Оптимизация алгоритмов:

- Прогнозирование спроса
- Балансировка нагрузки между водителями


Client
TaxiPark
DriverAssignmentStrategy
OrderSubject
Driver
request_ride()
select_driver()
driver
notify(driver)
update(order)
order + driver
Client
TaxiPark
DriverAssignmentStrategy
OrderSubject
Driver


Client
DriverInterface
TaxiPark
OrderSubject
Driver
PendingOrders
DriverAssignmentStrategy
Order
request_ride()
select_driver()
driver
notify(driver)
update(order)
order + driver
Процесс завершения поездки
complete_order(order_id)
update_status("completed")
set_available(true)
update_location(destination)
check_queue()
next_order
select_driver()
new_driver
notify(new_driver)
confirmation
Client
DriverInterface
TaxiPark
OrderSubject
Driver
PendingOrders
DriverAssignmentStrategy
Order


ClientInterface
- taxi_park: TaxiPark
+ init(taxi_park)
+ request_ride(client_id, pickup, destination) : Tuple[Order, int]
+ get_order_status(order_id: int) : str
DriverInterface
- taxi_park: TaxiPark
- driver_id: int
+ init(taxi_park, driver_id)
+ accept_order(order_id: int) : bool
+ complete_order(order_id: int)
+ set_availability(available: bool)
OwnerInterface
- taxi_park: TaxiPark
+ init(taxi_park)
+ add_driver(driver: Driver)
+ add_car(car: Car)
+ get_financial_report() : float
interacts with
interacts with
interacts with
TaxiPark
- drivers: List[Driver]
- cars: List[Car]
- orders: Dict[int, Order]
- pending_orders: List[Order]
- order_counter: int
+ create_order(client_id, pickup, destination) : Order
+ process_order(order: Order)
+ assign_driver(order: Order, driver: Driver)
+ complete_order(order_id: int)
+ check_pending_orders()
+ notify_drivers(order: Order, selected_driver_id: int)
+ get_available_drivers() : List[Driver]
+ add_driver(driver: Driver)
+ add_car(car: Car)
+ assign_driver_to_order(order_id: int) : int
+ get_order_status(order_id: int) : str
+ update_driver_availability(driver_id: int, available: bool)
+ calculate_total_earnings() : float
+ get_rating(driver_id: int) : float
+ change_driver_rating(driver_id: int, newRating: int)
DriverAssignmentStrategy
- _instance: DriverAssignmentStrategy
+ new() : DriverAssignmentStrategy
+ is_peak_hour() : bool
+ select_driver(drivers: List[Driver], order: Order) : Optional[Driver]
# calculate_distance(driver_x, driver_y, order_x, order_y) : float
contains
contains
represents
uses
uses
OrderSubject
- _observers: List[Driver]
+ attach(observer: Driver)
+ detach(observer: Driver)
+ notify(order: Order, selected_driver_id: int)
Car
+ car_id: int
+ model: str
+ license_plate: str
+ init(car_id, model, license_plate)
Order
+ order_id: int
+ client_id: int
+ pickup: MapPoint
+ destination: MapPoint
+ status: str
+ driver: Optional[Driver]
+ price: float
+ client_rating: float
+ init(order_id, client_id, pickup, destination, client_rating)
assigned to
pickup
destination
Driver
+ driver_id: int
+ name: str
+ is_available: bool
+ current_order: Optional[Order]
+ current_location: str
+ x: float
+ y: float
+ rating: float
+ init(driver_id, name, current_location, x, y, rating)
+ update(order: Order)
+ set_location(x, y)
+ end_order()
MapPoint
+ id: int
+ name: str
+ x: float
+ y: float
+ init(id, name, x, y)